ContractKV: Certified Prompt Compilation for Transactional Prefix Reuse in Multi-Tenant LLM Services

Bin Zhou and Xi Cao, *Member, IEEE*

***Abstract*—Prefix caching accelerates LLM serving only when reusable tokens form a contiguous physical prefix. In repeated agent, retrieval, and code services, 85.0% of prompt tokens are stable, yet native automatic prefix caching recovers only 2.4%. We present ContractKV, a service control plane that converts this latent stability into governed physical reuse. A contract-aware compiler constructs dependency-valid stable-prefix layouts; prospective qualification and strict teacher-forced attestation admit only behaviorally supported routes; signed, versioned certificates bind model, runtime, template, policy, authorization, and lifecycle identity; and Prefix-Selective Transactional Promotion makes only exact-LCP, block-aligned stable state visible to the shared cache. A frozen net-benefit gate retains raw order outside profitable scopes, while service-template certificates amortize in-engine verification across authorized tenants. On vLLM 0.10.2, four configurations from three model families reduce all-workflow mean TTFT by 29.41–44.69%, including raw fallback and measured verification charges. In a five-repeat Llama-3.1-8B comparison, ContractKV reduces mean TTFT by 29.16% over native APC, compared with 26.96% for CacheBlend over its matched runtime. Observed material paired task-score regressions are 0.089% overall and 0/1,125 among admitted compiled requests; both overall events occur on raw fallback and reproduce under raw-only control, whereas CacheBlend exhibits a 9.956% regression rate. In an independent same-engine Qwen3-8B closure, template-scoped ContractKV reduces mean TTFT by 92.90% after charging registration wall time. Across the 25-tenant load sweep it improves 500-ms SLO goodput by 4.81×, and at 80 tenants peak KV-block occupancy falls by 72%. Across live fault probes and 325,000 transactional stress operations, no aborted transaction publishes compiled blocks.**

***Index Terms*—KV cache, LLM inference serving, prefix caching, service-level objectives, multi-tenant consolidation, behavioral certification, transactional reuse.**

I. Introduction

Multi-tenant LLM inference services consolidate diverse agent, retrieval, and tool-augmented workflows onto shared GPU infrastructure, where service-level objectives (SLOs) on time-to-first-token (TTFT) and goodput govern deployment economics [1], [2]. We target deployments in which authorized tenants repeatedly consume a stable service template and its prefix remains resident often enough to amortize certification. Because prefill work grows with prompt length, modern serving engines widely deploy prefix caching: they physically reuse the key-value (KV) states of token-identical prefixes [1], [2], lowering TTFT without changing model semantics.

Prefix caching nevertheless has a structural blind spot in increasingly common structured services. Agent pipelines [14], retrieval-augmented generation (RAG), and multi-turn tool use [15] compose prompts from heterogeneous segments—system instructions, retrieved documents, repository context, observations, and conversation state—whose application order need not match their reuse potential. One changed observation between two stable segments terminates the physical prefix even when all downstream stable content is byte-identical. Across five representative service families, stable content accounts for 78–91% of prompt tokens, whereas native caching recovers under 3% in raw arrival order. The resulting gap between an *application-declared byte-stable working set* and *physical prefix continuity* creates redundant prefill work that compounds with context length and tenant fan-out.

Moving stable segments ahead of volatile ones can expose this reuse, but prefix maximization alone is not an admissible service policy. Segment order encodes data, causal, and positional dependencies; even a dependency-valid permutation can change position-sensitive generation. Once transformed KV state becomes visible in a shared cache, a behavioral error, stale identity, or scope mismatch can affect subsequent tenants.

Recent position-independent and cross-position reuse systems selectively compose, correct, or recompute moved segments [5]–[8]. They expand the reuse space but do not by themselves answer a service-level question: *under what evidence, identity, authorization, and transaction conditions may transformed state become shared service state?* A safe service must validate dependencies, bind model-backed evidence to runtime and authorization identity, revoke it across lifecycle changes, and prevent failed verification from partially publishing shared state. ContractKV therefore treats compiled KV as a versioned service object, not only a performance artifact.

We present ContractKV, a compile–attest–commit control plane for multi-tenant prefix reuse. (1) A *Contract-Aware Layout Compiler* (CALC) constructs a bounded ladder of dependency-valid layouts with progressively longer stable prefixes. (2) *Scoped Behavioral Certification* uses prospective qualification, strict teacher-forced token-path agreement, and signed versioned identity to admit only supported model–workflow–layout routes. (3) *Prefix-Selective Transactional Promotion* (PSTP) keeps candidate state private and makes

B. Zhou is with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 102218, China (e-mail: 2025200800@buct.edu.cn).

X. Cao is with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 102218, China (e-mail: caoxi@buct.edu.cn).

Corresponding author: Xi Cao.

only verified, exact-LCP, block-aligned stable blocks visible at one scheduler-thread linearization point. (4) A frozen *net-benefit gate* retains raw order outside profitable scopes, while a service-template certificate amortizes one deployment attestation across authorized tenants. Failure, identity mismatch, scope conflict, timeout, or profitability rejection publishes no compiled state.

We evaluate CONTRACTKV on vLLM 0.10.2 using Qwen3-8B, Qwen3-14B (TP=2), Mistral-7B-Instruct-v0.3, and Llama-3.1-8B. Across five workflows, all four configurations reduce all-workflow mean TTFT by 29.41–44.69% after accounting for direct L0 routing, runtime fallback, and measured verification charges. Under the same frozen RULER scope, Qwen, Mistral, and Llama converge to 85.97–87.41% reduction. In a five-repeat Llama comparison, ContractKV achieves a 29.16% matched-baseline reduction, compared with 26.96% for CacheBlend, while reducing observed material paired regressions from 9.956% to 0.089%; no such regression occurs among 1,125 admitted compiled requests. An independent same-engine verifier closure charges all template-registration wall time and reduces Qwen3-8B mean TTFT by 92.90%. Across the primary load sweep, ContractKV improves 500-ms SLO goodput by 4.81×, and at 80 tenants reduces peak KV-block occupancy by 72%. Live fault probes, signed-binding tests, and 325,000 state-machine transactions report no partial publication or stale-scope admission.

This paper makes the following contributions:

- We formulate *certified prompt compilation* for structured LLM services. An operator-registered manifest defines segment stability, separators, dependencies, authorization, and expected fan-out; undeclared tenant content defaults to private. CALC constructs a bounded ladder of dependency-valid native-prefix layouts.
- We introduce scoped behavioral certification and service-template certificates. Strict token-path attestation is bound to model/runtime identity, template version, policy, authorization domain, and lifecycle epoch, enabling deployment-level amortization without sharing private suffix state.
- We design PSTP, which keeps candidate KV state outside the shared prefix index until a valid commit and atomically publishes only the exact-LCP, block-aligned stable subset. Abort, timeout, duplicate, identity, and scope conflicts publish no compiled blocks.
- We implement ContractKV in the vLLM V1 scheduler and evaluate it across four model configurations, five task families, direct non-prefix-reuse baselines, same-engine verification, open-loop SLO workloads, 80-tenant consolidation, signed binding, live fault injection, and 325,000 transactional stress operations.

II. BACKGROUND AND MOTIVATION

A. Prefix Caching in LLM Serving

Modern LLM serving engines decompose inference into a **prefill** phase—which computes prompt KV states—and autoregressive **decode**. Their schedulers interleave both phases across requests. With *prefix caching*, a request sharing a resident token prefix reuses its KV blocks and computes only the suffix [1], [2].

Formally, let $\mathbf{t} = (t_1, \ldots, t_n)$ be a request's token sequence and $\mathcal{R}$ the set of resident KV states in the cache. The engine identifies the longest prefix match

$$k^* = \max\{k : (t_1, \ldots, t_k) \in \mathcal{R}\}, \tag{1}$$

reuses k^* blocks, and computes only the remaining $n - k^*$ tokens. Under *automatic prefix caching* (APC) in vLLM [1] or *RadixAttention* in SGLang [2], this matching is performed at block granularity B (typically 16 tokens), yielding physical reuse of $\lfloor k^*/B \rfloor$ blocks.

This mechanism is highly effective for *homogeneous* workloads—batch inference over identical system prompts, or multi-turn conversations where each turn appends to a growing prefix. In these settings, prefix reuse rates exceed 90% and TTFT reductions are substantial.

B. The Structural Blind Spot

Production LLM *services*, however, are increasingly *structured*: a single request composes multiple semantic segments—system instructions, retrieved documents, repository context, tool observations, and conversation state—whose physical ordering in the prompt is dictated by application logic rather than reuse potential. We term the resulting inefficiency the **structural blind spot**: the cache can only exploit *physical prefix continuity*, but the *semantic working set* that is actually reusable is distributed across non-contiguous positions.

Motivating measurement. We instrumented five representative service families (RULER, HotpotQA, NarrativeQA, Qasper, RepoBench) with 25 source-group-disjoint sessions each, 10 turns per session. Each session freezes source context into an operator-registered template and derives its later query/observation turns from a SHA-frozen trace manifest. For each request we compute:

- *Stable-token ratio* ρ_s: fraction of prompt tokens belonging to segments that remain byte-identical across turns within the same session.
- *Native prefix recovery* ρ_p: fraction of stable tokens actually recovered by the engine's native APC under raw arrival order.

Table I reports the results. The gap is stark: stable content constitutes 78–91% of prompt tokens, yet native prefix caching recovers fewer than 3% of this available reuse. The root cause is architectural: a single mutable segment (e.g., a tool observation) inserted between two large stable segments *fractures* the physical prefix, rendering all downstream stable content invisible to the cache despite being computationally identical.

This gap appears whenever an application interleaves byte-stable context with per-request observations and revisits the same authorized template. The degree of benefit is workload dependent: reuse must survive declared dependencies, behavioral admission, and cache residency. When those conditions hold, each unrecovered stable token incurs redundant prefill work, occupies scheduler slots, and consumes TTFT budget.

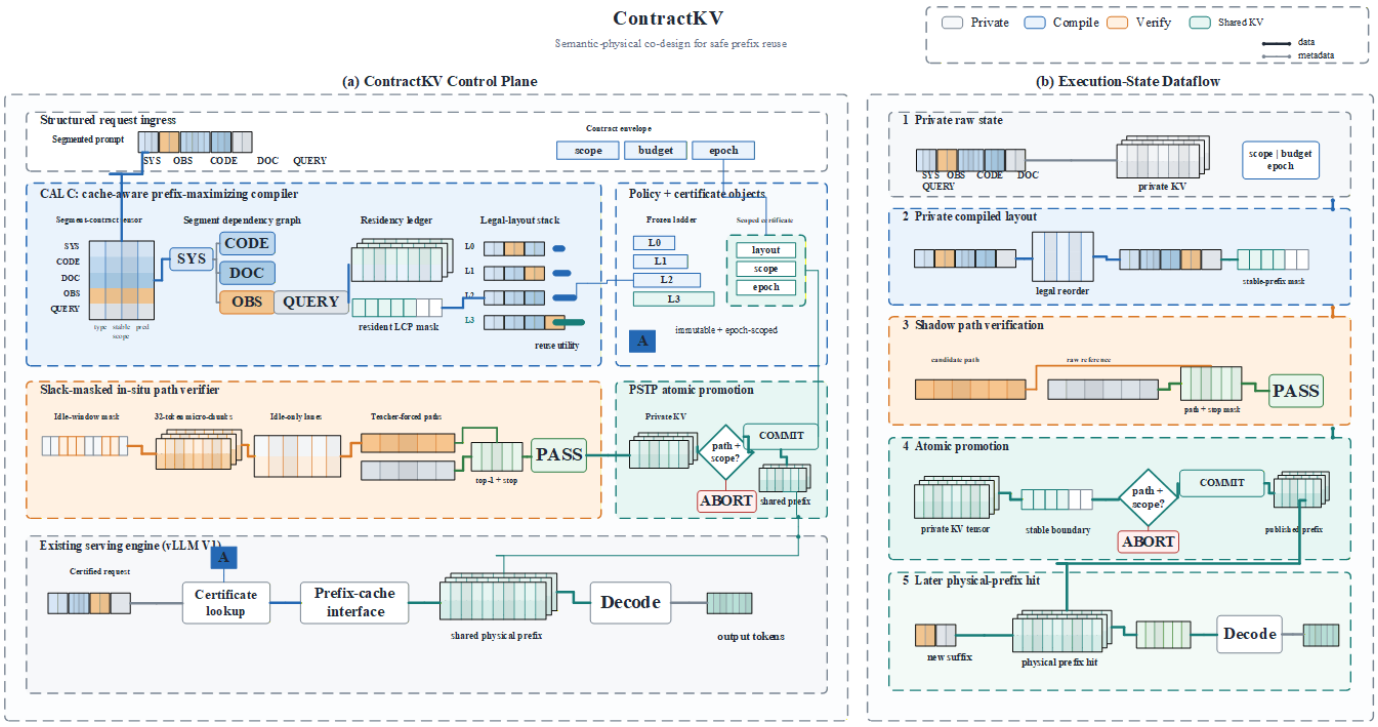


Fig. 1. ContractKV control-plane architecture and execution-state dataflow. (a) CALC combines segment contracts, the SDG, resident-prefix state, and qualification metadata; the verifier and PSTP publish only behaviorally certified compiled blocks into the vLLM cache domain. Solid arrows carry requests or KV state, whereas dashed arrows carry policy and certificate metadata. (b) A selected request evolves from private raw tokens to a private compiled layout, shadow verification, atomic promotion, and a later physical-prefix hit. A failed promotion discards only candidate compiled state; the raw-order path may populate native APC state as in unmodified vLLM.

TABLE I
STRUCTURAL BLIND SPOT: STABLE CONTENT VASTLY EXCEEDS PHYSICAL PREFIX RECOVERY ACROSS FIVE SERVICE FAMILIES (QWEN3-8B, 10-TURN SESSIONS).

Service Family	Stable Ratio ρ_s	Native Recovery ρ_p
RULER (NIAH)	91.2%	2.8%
HotpotQA	82.4%	2.1%
NarrativeQA	78.6%	1.9%
Qasper	84.1%	2.4%
RepoBench	88.7%	2.6%
Mean	**85.0%**	**2.4%**

C. Why Naive Reordering Is Inadmissible

A natural response is to *reorder* stable segments ahead of volatile ones, exposing a longer physical prefix. While this recovers reuse in principle, it introduces two correctness hazards that make unconstrained reordering unacceptable in a shared serving system:

Hazard 1: Dependency violation. Segment order encodes semantic dependencies. A tool observation may depend on a preceding system instruction; a code completion requires repository context before the query; a chronological agent history must preserve causal ordering. Arbitrary permutation violates these constraints, producing prompts that are cache-friendly but *semantically ill-formed*.

Hazard 2: Behavioral drift. Even when all dependencies are satisfied, a reordered prompt may shift the model's output distribution. Transformer attention is position-sensitive: moving a segment from position i to position j changes its attention pattern and can alter generation. Publishing an unverified reordered prefix into a shared cache makes this behavioral consequence *visible to all subsequent requests*—a silent, system-wide correctness risk with no rollback mechanism.

The fundamental tension. Exploiting structural reuse requires layout control; safe layout control requires verification; and reusable verification requires explicit scope and transactional publication. To our knowledge, no prior system jointly provides these properties. Figure 1 shows how ContractKV connects the compiler, certifier, and cache publication path.

III. SERVICE MODEL AND PROBLEM FORMULATION

We now formalize the structured-prompt prefix-maximization problem that ContractKV solves. The formulation proceeds in three layers: a segment model with typed stability annotations, a dependency-constrained layout optimization objective, and a multi-tenant service-template abstraction.

A. Structured Request and Segment Model

Definition 1 (Structured Request): A **structured request** r is an ordered tuple of *semantic segments*

$$r = (s_1, s_2, \ldots, s_m), \quad s_i = (id_i,\ \tau_i,\ \sigma_i,\ \mathbf{c}_i), \tag{2}$$

where id_i is a unique segment identifier, $\tau_i \in \mathcal{T} = \{\text{STABLE}, \text{MUTABLE}, \text{QUERY}\}$ is the **stability type**, σ_i is

a content hash, and $\mathbf{c}_i$ is the token sequence of segment s_i. Stability types and legal dependencies are supplied by an operator-registered service-template manifest; unregistered tenant inputs default to private mutable or query segments.

Trust boundary. STABLE is a privileged type granted only by an operator-registered, versioned service manifest. Request-time tenant input, tool output, request identifiers, and undeclared segments default to private MUTABLE or QUERY state and cannot acquire shareable status through request metadata. The manifest binds segment boundaries, frozen separators, content hashes, dependency edges, and the authorization domain.

The stability type captures the segment's reuse potential within a service session:

- STABLE: content is byte-identical across all turns in the session (system prompts, retrieved documents, repository context). These segments constitute the *reusable working set*.
- MUTABLE: content changes across turns but follows a predictable lifecycle (tool observations, intermediate results). These segments *fracture* prefix continuity.
- QUERY: the per-turn user query or action. Always unique; never reusable.

Let $L(r) = \sum_{i=1}^{m} |\mathbf{c}_i|$ denote the total prompt length and $L_s(r) = \sum_{i:\tau_i=\text{STABLE}} |\mathbf{c}_i|$ the stable-token length. The *structural reuse potential* is $\rho_s(r) = L_s(r)/L(r)$.

B. Segment Dependency Graph

Definition 2 (Segment Dependency Graph): The **segment dependency graph** (SDG) is a DAG $G_r = (V, E, \lambda)$ over segments $V = \{s_1, \ldots, s_m\}$. An edge $(s_i, s_j) \in E$ requires s_i to precede s_j; $\lambda : E \to \{\text{DATA}, \text{CAUSAL}, \text{POSITIONAL}\}$ records why.

Edge types carry distinct semantics: DATA edges encode information flow (a tool result depends on its input); CAUSAL edges encode temporal ordering (turn k depends on turn $k-1$); POSITIONAL edges encode model-specific sensitivity (instruction must precede query). The SDG is constructed from the operator-registered manifest's segment boundaries, content hashes, and dependency edges; it does not infer shareability from tenant-provided metadata.

Definition 3 (Layout): A **layout** π of request r is a permutation of V such that π is a *topological ordering* of G_r:

$$\forall (s_i, s_j) \in E: \quad \text{pos}_\pi(s_i) < \text{pos}_\pi(s_j). \tag{3}$$

The set of all valid layouts is $\Pi(G_r) = \{\pi : \pi \text{ is a topological order of } G_r\}$.

C. Layout Validity and Prefix Maximization

Given a layout π, the *compiled prompt* is the concatenation $\mathbf{c}_\pi = \mathbf{c}_{\pi(1)} \| \mathbf{c}_{\pi(2)} \| \cdots \| \mathbf{c}_{\pi(m)}$. Let $q(\pi)$ be the number of leading stable segments and $P(\pi)$ their token length:

$$\begin{aligned} q(\pi) &= \max\{q : \forall j \le q,\ \tau_{\pi(j)} = \text{STABLE}\}, \\ P(\pi) &= \sum_{j=1}^{q(\pi)} |\mathbf{c}_{\pi(j)}|. \end{aligned} \tag{4}$$

The physical reuse at block granularity B is $\lfloor P(\pi)/B \rfloor$ blocks.

Problem 1 (Dependency-Constrained Prefix Maximization): Given a structured request r with SDG G_r, resident state $\mathcal{R}$, and a finite compiler ladder $\Pi_L(G_r) = \{\pi_0, \ldots, \pi_K\} \subseteq \Pi(G_r)$, maximize block-aligned reuse over the frozen ladder:

$$\pi^* = \arg \max_{\pi \in \Pi_L(G_r)} \textit{reuse}_B(\pi, \mathcal{R}) \tag{5}$$

$$\text{s.t.} \quad \textit{BehavioralAdmission}(\pi, r; \theta, C) = \text{PASS}, \tag{6}$$

$$\textit{ScopeValid}(r, C) = 1, \tag{7}$$

$$\textit{BenefitGate}(\pi, \Theta) = \text{PASS}. \tag{8}$$

Here π_0 is raw order and each ladder member is dependency-valid by construction. The compiler maximizes reuse only within the set admitted by three frozen controls: behavioral attestation, certificate scope, and prospective net benefit. The implementation takes the first passing member in frozen high-to-low order; resident state is a reuse witness and admission input, not an online optimizer of construction order. The 0.03 task-score threshold was frozen on development data before sealed evaluation as the uniform material-effect reporting threshold across task scorers; it is a post-hoc quality gate, not an admission tolerance or proof of equivalence.

Exact optimization would search dependency-valid topological orders while evaluating a model-backed behavioral predicate. ContractKV instead uses a bounded *layout ladder* (Section IV-A); the ledger rejects candidates that expose no resident block and certification evaluates the remaining frozen-order candidates. We do not rely on an unproven complexity classification for this search problem.

D. Service-Template Model

In multi-tenant LLM serving, multiple tenants issue requests against a shared stable context. We abstract this as:

Definition 4 (Service Template): A **service template** $\Theta = (\mathbf{S}, \mathcal{A}, \mathcal{P}, \mathcal{M}, F)$ binds ordered stable segments $\mathbf{S}$, authorized tenants $\mathcal{A}$, frozen policy $\mathcal{P}$, operator manifest $\mathcal{M}$ (boundaries, dependencies, separators, and domain), and declared authorized fan-out F used by the benefit gate.

Each tenant a_i appends private MUTABLE and QUERY segments to $\mathbf{S}$, forming a tenant-specific request $r_i = \mathbf{S} \| \mathbf{m}_i \| \mathbf{q}_i$. The **service locality** of a workload is the expected number of tenants sharing each template; higher locality implies greater amortization of verification cost and prefix-state residency.

The service-level objective extends Problem 1 to the multi-tenant setting: maximize aggregate SLO goodput (fraction of requests meeting a TTFT deadline Δ_{SLO}) across all tenants, subject to unauthorized-reuse isolation (private suffix state is not shared across tenants) and certificate-scope admissibility.

E. Safety and Isolation Invariants

ContractKV enforces four design invariants in the implementation and validates their failure paths through source-level instrumentation and fault injection:

(P1) Publication safety. No *candidate compiled* KV state becomes visible in the shared prefix cache unless it has passed

behavioral verification *and* carries a valid certificate whose scope subsumes the current request:

$$\mathit{Visible}_{\mathrm{compiled}}(x) \implies \exists\, C : \mathit{Valid}(C, x) \wedge C.\mathit{status} = \textsc{Pass}. \tag{9}$$

(P2) Suffix isolation. The transactional commit boundary is block-aligned to the stable-prefix length and exact candidate–certificate token LCP; no MUTABLE or QUERY token is promoted:

$$\begin{aligned} L_{\mathrm{lcp}} &= \mathrm{LCP}(\mathbf{t}_{\mathrm{cand}}, \mathbf{t}_{\mathrm{cert}}), \\ L_{\mathrm{commit}} &= B \left\lfloor \frac{\min(L_s, L_{\mathrm{cert}}, L_{\mathrm{lcp}})}{B} \right\rfloor, \\ N_{\mathrm{blocks}} &= L_{\mathrm{commit}}/B. \end{aligned} \tag{10}$$

where L_{cert} is the prompt-prefix length authorized by attestation.

(P3) Fail-closed semantics. Any verification failure, scope mismatch, timeout, or identity conflict aborts promotion with *zero publication of candidate compiled state*. The request then resumes under raw order; assuming progress of the base engine, this fallback also preserves progress. Native APC may populate raw-order cache state exactly as in unmodified vLLM.

(P4) Tenant isolation. For tenants $a_i \neq a_j$ under the same template Θ, private observations, request identifiers, and candidate namespaces remain disjoint. Cross-tenant KV reuse is confined to the authorized stable prefix $\mathbf{S}$; this is a cache-state isolation claim, not a claim about the surrounding application's security boundary.

IV. CONTRACTKV DESIGN

ContractKV resolves Problem 1 through a four-stage pipeline—*compile*, *certify*, *promote*, *amortize*—that operates entirely within the serving engine's existing prefix-cache substrate. No hardware modification, custom kernel, or external oracle is required. We present each stage in turn.

A. Contract-Aware Layout Compiler

The **Contract-Aware Layout Compiler** (CALC) transforms request r into a bounded ladder of dependency-valid candidates. "Contract-aware" is deliberate: cache residency informs benefit admission, while semantic dependencies, authorization, and frozen qualification determine legality. L0 is raw order. For Llama calibration, stable content is split into $K = 16$ contiguous character-preserving units; L1/L2/L3 select 4/12/16 eligible units. A stable-priority topological sort moves units only as early as declared SDG edges permit, breaks ties by original index, inserts frozen separators, and requires full retokenization plus an exact rendered-prompt audit. Qwen and Mistral evaluate one full candidate, L3*, under the same certificate protocol; they do not claim separately calibrated L1/L2 semantics.

Prospective qualification. For a new model, every family–level pair is evaluated on a source-disjoint development split. The highest level satisfying completion, future-turn quality, scope, output-length, physical-reuse, and suffix-isolation gates becomes the frozen family maximum. The eligible descent set, policy hash, and qualification hash are frozen *before* the sealed held-out split is opened. Held-out outcomes never change the ladder or thresholds.

Algorithm 1 Frozen certificate-guided layout descent.

```
Validate manifest; default undeclared segments to private
Split stable text; build L0–L3 by stable-priority topological sort
Break ties by original index; retokenize and audit each prompt
if frozen benefit gate rejects the declared fan-out then
    return raw order L0
end if
for all ℓ in frozen eligible levels, high to low do
    Compile π_ℓ; require one added block; run compact attestation
    if attestation is PASS and scope is valid then
        PSTP commit; return π_ℓ
    end if
end for
return raw order L0 with no shared-state publication
```

Runtime descent and benefit admission. The compiler tries only levels in the frozen eligible descent set. Before verification, a frozen minimum-benefit rule rejects scopes without sufficient certified fan-out; each remaining candidate must add at least one reusable block and pass strict verification and scope validation. Any failed gate descends, eventually returning L0. Algorithm 1 is the implemented admission path.

The topological construction costs $O(m \log m)$ for m segments. Qualification is lightweight in our measured onboarding: 25 development sessions over five families and four ladder cells consume 0.359 GPU-hours; the frozen manifest is 1,678 B and runtime lookup is 0.125 μs mean (0.200 μs p95). The frozen gate accepts 2/4 sealed operating points; both accepted lanes are net-positive, with a minimum observed benefit of 7.04 s per certificate. The two rejected lanes execute raw order, and their diagnostic candidate outcomes are not used to revise the frozen rule.

B. Scoped Behavioral Certification

A dependency-valid layout is *necessary but not sufficient* for safe publication. Transformer attention is position-sensitive: permuting segments changes their absolute and relative positions, potentially shifting the output distribution even when all dependencies are satisfied. ContractKV therefore requires *empirical behavioral certification* before any compiled layout enters the shared cache.

In-engine teacher-forced verification. All certified serving paths use deterministic greedy decoding with temperature 0 and stochastic sampling disabled. The generation configuration, stop protocol, output budget, model/runtime identity, and tokenizer/template identity are certificate-bound. Given candidate and raw prompts, the verifier first obtains the raw reference output path $\mathbf{y} = (y_1, \ldots, y_{L_y})$. It then evaluates both prompts on the same teacher-forced history $\mathbf{y}_{<i}$. The admission contract is **strict top-1 agreement** along this generation path:

$$\begin{aligned} \forall i \le L_y : \quad & \arg\max p_\theta(\cdot \mid \mathbf{c}_{\pi^*}, \mathbf{y}_{<i}) \\ & = \arg\max p_\theta(\cdot \mid \mathbf{c}_{\pi_{\mathrm{raw}}}, \mathbf{y}_{<i}). \end{aligned} \tag{11}$$

If every position agrees and the output path completes under the frozen task budget, the verifier emits PASS; otherwise it fails closed. L_y is the verified output-path length. The distinct value L_{cert} is the stable prompt-prefix length authorized by

that pass. Thus strict top-1 agreement certifies the frozen deterministic path; it does not claim distributional equivalence under sampling. The 0.03 task-score gate is a separately frozen post-hoc material-regression threshold, not a relaxed admission tolerance.

Compact attestation. The verification result is transmitted as a *compact attestation* rather than a full token-by-token trace:

$$\mathcal{A} = (\textit{decision},\ \textit{tx_id},\ L_{\mathrm{cert}},\ \textit{policy_hash},\ \textit{timestamp}). \quad (12)$$

This five-field record keeps per-token logprob tensors off scheduler egress. A controlled wire measurement is 387 B versus 831,262 B for the corresponding full response (2,148× smaller; largest core record 231 B). This is an implementation result, not the basis of the latency headline.

Certificate scope binding. Let $I = H(W, M, T_k, G, R)$ bind weight/checkpoint W, model configuration M, tokenizer/chat template T_k, frozen generation protocol G, and serving runtime R. A behavioral certificate is a signed empirical attestation:

$$C = \langle I, V, H_s, L, P, Q, A, E, \Sigma \rangle, \quad (13)$$

Here V is immutable template version, H_s stable-content hash, L layout hash, P verifier policy, Q qualification-manifest hash, A authorization domain, E lifecycle epoch, and Σ the issuer signature. The run manifest binds CUDA runtime, dtype, tensor parallelism, output budget, stop protocol, sampling parameters, and seed policy. Reuse requires exact identity and scope subsumption; any mismatch returns raw. A 15-case signed-binding stress has zero false admit/reject: unseen private suffixes remain admissible, while malicious metadata fails closed.

Evidence and scope layers. ContractKV separates three layers often compressed into "certificate": (i) source-disjoint route qualification selects a model–workflow–layout scope; (ii) immutable template registration attests one deployment object and publishes after PSTP; and (iii) every reuse checks identity, lifecycle, and authorization. Runtime scopes are OBSERVATION (exact content/epoch), QUALIFIED-FAMILY (frozen level/descent set), and SERVICE-TEMPLATE (one immutable deployment/domain). They are empirical scopes, not universal claims over arbitrary inputs.

C. Prefix-Selective Transactional Promotion

Even with a valid certificate, the act of publishing KV state into a shared prefix cache must be *transactional*: either the entire verified stable prefix becomes atomically visible, or nothing changes. ContractKV enforces this through **Prefix-Selective Transactional Promotion** (PSTP).

Block-aligned commit boundary. Given block granularity B, stable-token boundary L_s, authorized boundary L_{cert}, and the exact candidate–certificate token LCP, PSTP computes:

$$L_{\mathrm{lcp}} = \mathrm{LCP}(\mathbf{t}_{\mathrm{cand}}, \mathbf{t}_{\mathrm{cert}}),$$
$$\textit{Commit}(r) = B \left\lfloor \frac{\min(L_s, L_{\mathrm{cert}}, L_{\mathrm{lcp}})}{B} \right\rfloor. \quad (14)$$

Thus publication cannot cross the stable boundary, the authorized boundary, or an exact token mismatch; the floor operation excludes partial blocks.

Ephemeral candidate state. Between compilation and commit, all KV states for the candidate layout reside in an *ephemeral namespace* invisible to the prefix-cache index. Other requests cannot observe, match against, or evict this state. Only upon successful commit does the state migrate to the shared namespace and become eligible for prefix matching.

State machine and abort paths. PSTP advances candidate state through PRIVATE → VERIFYING → VERIFIED → COMMITTED; any failure transitions to terminal ABORTED. The sole linearization point is insertion of certified block hashes into the shared index. PSTP defines five abort conditions, each resulting in *zero publication of candidate compiled state*:

1) NON-PASS: verification returns FAIL or remains incomplete;
2) DUPLICATE: a transaction with the same *tx_id* already committed;
3) TIMEOUT: verification exceeds the frozen hard deadline $\Delta_{\mathrm{deadline}}$;
4) IDENTITY MISMATCH: model, policy, or layout hash differs from the certificate;
5) SCOPE INVALID: the request falls outside the certificate's validity scope.

Upon abort, candidate compiled state is discarded and the request proceeds under raw-order semantics. The system never publishes partially verified or scope-invalid compiled blocks. Raw execution can subsequently populate native APC state, so the invariant is zero *candidate publication*, not zero change to the entire cache domain. Control-plane work may still add latency—the zero-locality negative control measures this overhead explicitly—so the guarantee is behavioral and transactional, not performance monotonicity.

D. Service-Template Certificate

In multi-tenant LLM serving, per-session verification scales with the number of tenants. The **service-template certificate** combines prospective route qualification with deployment attestation for one immutable template version. It is not a per-query equivalence proof: it authorizes a prospectively qualified model–workflow–layout route for one immutable template and runtime identity. Subsequent requests are admitted by complete identity and authorization checks; empirical validity over unseen private suffixes is evaluated separately.

Template registration. When a qualified service template $\Theta = (\mathbf{S}, \mathcal{A}, \mathcal{P}, \mathcal{M}, F)$ is deployed, ContractKV attests stable content $\mathbf{S}$ under authorization $\mathcal{A}$, policy $\mathcal{P}$, manifest $\mathcal{M}$, complete runtime/model identity I, and operator-declared future fan-out F. It publishes C_Θ only after PSTP succeeds.

Request-scope check. For each authorized request $r_i = \mathbf{S} \| \mathbf{m}_i \| \mathbf{q}_i$, the scope check requires stable-content hash equality, tenant authorization, and the complete composite identity I (weights/configuration, tokenizer/template, generation and

stop/output protocol, and runtime fingerprint). Template version, qualification, policy, manifest, layout, and lifecycle epoch must also match C_Θ. A pass reuses the certified prefix without per-request verification; Section VI-E measures this bounded hash/authorization path rather than assuming it free.

Lifecycle invalidation. Model-weight, tokenizer, policy, template-content, authorization, or administrative-revocation changes invalidate the certificate and evict dependent prefix state. The next request must re-register, preventing reuse beyond the empirical identity boundary.

Amortization analysis. For N authorized sessions or tenants sharing one template and T turns per session, session scope performs $O(N)$ full attestations plus $O(NT)$ certificate lookups. Template scope performs $O(1)$ full attestation plus the same $O(NT)$ bounded lookups. At 25-tenant locality, this change of certificate object lowers measured unhidden verification cost from 25.05 to 3.24 ms/request (7.7×) without materially changing new-prefill work (Section VI-E).

V. Implementation

We implement ContractKV as a middleware layer atop **vLLM 0.10.2** [1], targeting the V1 scheduler path. The implementation comprises three components: a layout compiler integrated at the request-admission boundary, a verification executor that shares the model backend, and a transactional promoter hooked into the prefix-cache block allocator.

Scheduler integration. The layout compiler intercepts each incoming request before scheduler admission. It constructs the SDG from the operator-registered template manifest, fully retokenizes the rendered candidate, and emits the compiled token sequence. The serving interface treats only manifest-declared stable slots as shareable; user input, tool output, and request identifiers remain private. ContractKV does not modify attention kernels, model weights, or sampling logic. It adds scheduler-side admission hooks and cache-index publication logic to the vLLM V1 block-management path.

Physical prefix-hit and new-prefill token counts are recorded at the scheduler's block-allocation boundary. For each request, the scheduler reports the number of blocks served from the resident prefix cache versus blocks requiring fresh prefill computation. These counters provide ground-truth physical reuse metrics independent of timing inference.

Verification executor. The behavioral verifier reuses the serving model backend and weight instance. In the same-engine closure, verifier registrations enter the same live vLLM service path as foreground requests; all registration wall time is charged to end-to-end goodput. Five fresh processes each issue and commit four template registrations with zero registration failure. Historical broad cross-model protocols execute session verification in a soft hiding window Δ_{slack} and conservatively charge residual user-visible exposure as

$$C_{\text{unhidden}} = \max(C_{\text{verify}} - \Delta_{\text{slack}}, 0). \tag{15}$$

Crossing this soft window does not abort verification. A separate frozen hard timeout enforces $C_{\text{verify}} > \Delta_{\text{deadline}} \Rightarrow$ ABORT. Qwen3-8B, Mistral, and Qwen3-14B freeze 2,000 ms slack; broad Llama qualification freezes 6,000 ms. These historical rows use explicit residual-cost accounting, not a claim of free overlap; same-engine D2 is the deployment-path closure.

TABLE II
FROZEN PRIMARY PROTOCOLS. QUANTILES POOL REQUESTS WITHIN A FRESH PROCESS AND ARE SUMMARIZED ACROSS INDEPENDENT PROCESSES.

Study	Per repeat	Total/cell	Output
All-workflow / common scope	450 / 270	1,350 / 810	task-specific
Llama direct baselines	450	2,250	96 generated
Same-engine D0/D1/D2	450	2,250	96 generated
Open-loop / consolidation	375	1,125	64-token marker
Template-quality audit	450 paired	1,350 paired	original budgets

The compact attestation is serialized as a fixed-size record and stored in an in-memory certificate table indexed by the nine-field versioned key. The source-level egress path emits no prompt-logprob tensor; request and transaction identity remain scheduler-local. Certificate lookup uses a hash map.

Transactional promoter. PSTP is implemented as a commit gate in the prefix-cache block-management path. Candidate KV blocks remain excluded from shared prefix matching until a valid PASS. The scheduler thread then calls the cache manager's registration path for exactly $N_{\text{blocks}} = L_{\text{commit}}/B$ certified blocks. This insertion of certified block hashes into the shared prefix index is the linearization point; the KV tensors are already resident and are not copied during commit. On abort, candidate registration is skipped and native cleanup proceeds.

Fresh-process experimental framework. To eliminate cross-cell cache contamination, our evaluation harness creates *independent server processes* for each experimental cell. Each cell receives a dedicated GPU assignment, port, compile cache, and metrics log. The request runner alternates policy order (raw-order vs. ContractKV) and GPU assignment across repetitions to control for thermal and scheduling bias. Workload traces are SHA-frozen before measurement; partial cell resume is forbidden. This framework ensures that reported metrics reflect steady-state serving behavior rather than warm-start artifacts.

Compiler and protocol overheads remain below model-serving latency: CALC is 0.389 ms mean (0.819 ms p95), residency lookup 0.249 ms (0.507 ms), template-scope check 0.109 ms (0.111 ms), and PSTP finalization 0.636 ms (0.685 ms).

VI. Evaluation

We evaluate ContractKV across four model configurations from three families, five task workflows, and multi-tenant service deployments. Primary task-quality and common-scope experiments require 100% completion, zero policy-attributable quality violations, zero scope-invalid admissions, and zero unauthorized cross-tenant KV reuse. Open-loop service cells instead report measured completion and drop rates explicitly.

A. Experimental Setup

Hardware and software. One server with two NVIDIA RTX 3090 GPUs (24 GB each), 128 GB system RAM, running

TABLE III
GLOBAL ALL-WORKFLOW RESULTS (3 FRESH-PROCESS REPETITIONS; RAW FALLBACK AND RESIDUAL VERIFICATION EXPOSURE INCLUDED). EACH MODEL–WORKFLOW CELL USES ONE DEVELOPMENT-FROZEN SLO APPLIED IDENTICALLY TO RAW AND CONTRACTKV; DEADLINES MAY DIFFER ACROSS CELLS.

Metric	Qwen 8B	Qwen 14B	Mistral	Llama
Net mean TTFT red.	37.81%	44.69%	31.72%	29.41%
New-prefill red.	59.13%	60.57%	39.46%	30.97%
Net SLO goodput (×)	2.64	2.92	1.24	1.19
Cert. coverage	68%	68%	50%	50%
Quality violations	0	0	0	0
Scope-invalid adm.	0	0	0	0
Completion	100%	100%	100%	100%

vLLM 0.10.2 with automatic prefix caching enabled. Qwen3-8B, Qwen3-14B, and Mistral use an 8,192-token limit; the broad Llama protocol uses 16,384 tokens. Qwen3-14B uses tensor parallelism 2; the other models use one GPU. Primary paired serving uses concurrency 2.

Models. Four configurations spanning three model families and the 7–14B parameter class: **Qwen3-8B** (primary), **Qwen3-14B** (TP=2 scale-up), **Mistral-7B-Instruct-v0.3**, and **Llama-3.1-8B-Instruct** (adaptive qualified policy). Each model uses a prospectively frozen layout policy with no cross-model threshold transfer.

Workload. Five service families (RULER [16], HotpotQA, NarrativeQA, Qasper, RepoBench) span long-context retrieval, reasoning, and code [19]. Each model evaluates 25–50 sealed sessions *in total*, balanced across the five families, with 10 turns per session and three fresh-process repetitions.

Protocol. Policy order and GPU assignment are rotated across repetitions. We report *net* mean TTFT (observed TTFT plus C_{unhidden}), new-prefill tokens, SLO goodput, and certificate coverage. Each model–workflow cell uses one development-frozen SLO deadline applied identically to Raw and ContractKV; deadlines may differ across model–workflow cells. Task-specific output budgets are frozen from development profiling; the 64-token deterministic marker used in service-load cells isolates queueing, prefix reuse, and TTFT and is not a production output cap. Full task metrics are measured in separate paired quality runs. Request-level quantiles are pooled within each fresh process; reported repeat summaries and session-cluster bootstrap confidence intervals preserve process and paired-session clusters.

B. Global All-Workflow Results

Table III and Fig. 2 report the primary all-workflow results across four configurations. Every global row includes raw fallback and residual verifier exposure; ContractKV remains net-positive while preserving the frozen quality and scope gates.

Qwen3-8B is the primary service platform because it supports the complete deployment matrix; Section VII-A reports its strongest end-to-end result. Qualification yields positive global gains for every configuration across three families and two parameter scales. New-prefill reduction exceeds net TTFT reduction (e.g., 59.13% vs. 37.81% for Qwen3-8B), exposing serving and verification cost already charged by

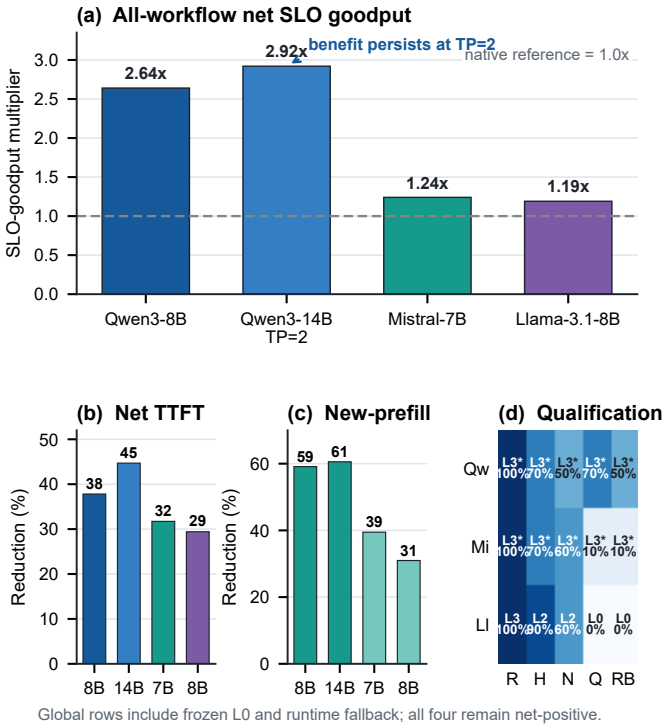


Fig. 2. Global benefit and qualified scope. (a) Net SLO-goodput improves by 1.19–2.92×. (b–c) Net mean TTFT and new-prefill work fall by 29.41–44.69% and 30.97–60.57%. (d) Each cell prints its frozen level and measured certificate coverage; L0 denotes raw order, while L3* is the full compiled candidate without separately calibrated L1/L2.

the net metric. In a same-runtime decomposition, Qwen3-8B measures 1,396.4 ms under raw APC, 623.0 ms with the frozen layout, and 868.4 ms with full online ContractKV; Mistral measures 1,102.5, 695.8, and 752.8 ms. Qwen3-14B TP=2 reaches 44.69% net TTFT and 60.57% prefill reduction, retaining and strengthening the measured benefit at a larger configuration without asserting a cluster-scale law.

Coverage and fallback accounting. The global net metric includes compiled requests, runtime fallback, families prospectively frozen at L0, and verification time not hidden by slack. Thus 50% coverage on Mistral and Llama does not mean that half of requests perform a failed verification: unqualified families route directly to L0 under the frozen manifest. Coverage is a measured model–workflow admission outcome, not an unaccounted confound. The only materially negative compiled cell is Mistral RepoBench at 10% sparse coverage (-2.22% net TTFT); L0 remains within approximately $\pm 1\%$ raw-run noise. A universal aggressive layout is therefore unsafe, whereas certification exposes each model's maximum admissible structural reuse.

C. Cross-Model Convergence on Common Scope

A critical question is whether ContractKV's gains are model-specific or converge under identical conditions. We answer this with a *common frozen RULER scope*: 30 source-group-disjoint sessions replayed on three models under the same L3 layout, with no requalification.

Table IV shows close convergence: the macro TTFT reduction is **86.67%** (95% CI [86.44, 86.90]) and the estimates

TABLE IV
COMMON FROZEN RULER SCOPE: THE SAME 30 FROZEN SESSIONS ACROSS MODELS, 10 TURNS, AND 3 FRESH-PROCESS PAIRS PER MODEL (TRACE SHA-256 COMMITTED).

Metric	Qwen 8B	Mistral	Llama
Net mean TTFT red.	86.64%	87.41%	85.97%
95% CI	[86.16, 87.10]	[87.06, 87.79]	[85.63, 86.29]
New-prefill red.	96.71%	95.86%	95.45%
SLO req/s (raw→CKV)	2.42→6.33	2.67→8.31	3.42→14.42
Goodput multiplier	2.61×	3.11×	4.22×
Cert. coverage	100%	100%	100%
Quality violations	0	0	0
Paired outputs	810	810	810

lie within 1.44 percentage points. The same sealed sessions, manifest, L3 layout, turns, and fresh-process protocol remove sample and layout asymmetry. With 100% certified coverage, this separates *model-dependent admission coverage* from the *reusable-prefix mechanism*: equal certified scope yields nearly equal redundant-prefill recovery.

The SLO-goodput multiplier varies (2.61–4.22×) because each model has a different raw decode rate; this reflects baseline throughput differences, not mechanism strength. As a portability check, the same offline-frozen safe layouts reduce real SGLang 0.4.9 RadixAttention TTFT by 54.93% on Qwen and 35.53% on Mistral. This shows that another prefix substrate can consume compiler output; it is not full online ContractKV because SGLang does not run PSTP or issue certificates.

D. Direct Comparison with Non-Prefix Reuse

Fig. 3(a) compares runtime-native implementations using the same Llama-3.1-8B weights, hardware, sealed workload, concurrency, and output budget. Because systems retain their native serving and prompt-serialization stacks, causal comparisons use each method's delta against its matched baseline. ContractKV and CacheBlend reduce mean TTFT by 29.16% and 26.96%, respectively, while their observed paired > 0.03 regression rates are 0.089% and 9.956%; none of 1,125 admitted compiled requests regresses. Their absolute means are 722.92 and 739.04 ms, but this descriptive difference is not treated as a cross-runtime causal comparison. The manual row is an unsafe offline upper bound; SGLang consumes an offline-frozen layout but performs neither online qualification nor PSTP.

Fig. 3(b) localizes the tail: compiled requests have p99 899 ms and 100% attainment at 1,500 ms, whereas the aggregate p99 is concentrated in deliberate L0. Across all 2,250 paired ContractKV outputs, 2 material regressions (0.089%) occur on raw fallback and reproduce under raw-only control. The admitted compiled subset has 0/1,125 events (one-sided 95% exact upper bound 0.266%). ContractKV is the only measured deployable row combining greater than 29% matched reduction with zero observed admitted-path regressions.

Runtime controls. The vLLM pair averages 2,471.54 prompt tokens, while the native LMCache pair averages 2,300.08 because it retains its separator serialization. CacheBlend uses LMCache 0.3.9 and FlashInfer 0.3.1 on CUDA 12.8; compatibility changes are limited to FP16 self-test tensors, CUDA

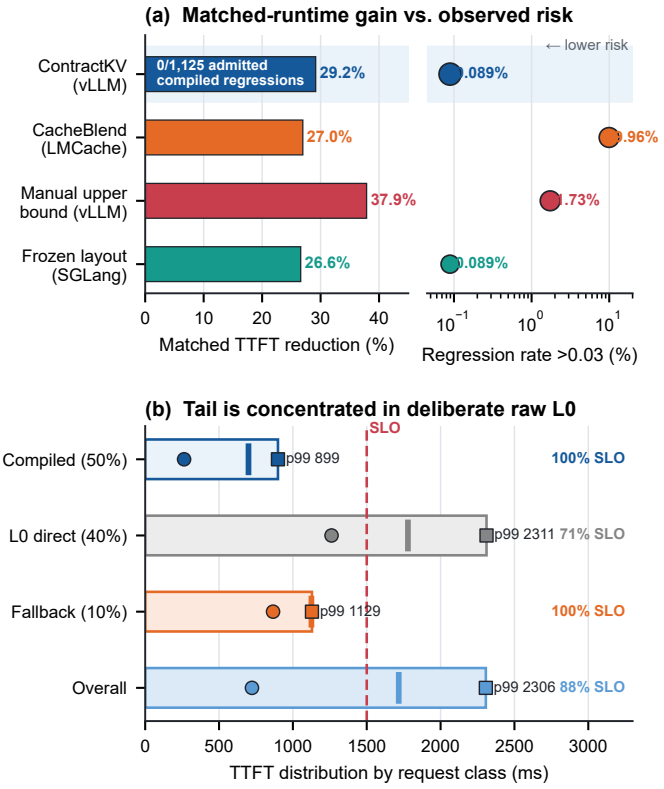


Fig. 3. Llama comparison and tail attribution (five fresh processes). (a) ContractKV and CacheBlend reduce mean TTFT by 29.16% and 26.96% relative to their respective matched runtimes; observed paired material-regression rates are 0.089% and 9.956%. Manual is an unsafe upper bound; SGLang consumes only a frozen layout. (b) Bars end at p99, with mean circles and p95 ticks; the tail is concentrated in raw L0.

JIT routing, and a 32-token chunk required by its layerwise incomplete-chunk rule. Its regression rate is computed against its own separator-only baseline. The supplied CacheSlide artifact executes after a Llama RoPE backport but retains process-global request state and fails output equivalence at concurrency 2; we report the blocker rather than a proxy performance row.

E. Service-Template Certificate

The service-template certificate is ContractKV's most distinctive service-computing contribution. We validate it along three axes: same-engine path closure, quality preservation under fan-out, and verification amortization.

Same-engine path closure. We replay identical five-repeat open-loop traces through D0 (native raw APC), D1 (the frozen safe layout without online verification), and D2 (full ContractKV with verifier registration and certificate publication in the serving path). Fig. 4(a–b) shows that D2 preserves D1's benefit: 500-ms SLO goodput is 1.428 versus 1.472 requests/s, mean TTFT is 194.72 versus 194.95 ms, and both have 935.86-ms p99. D2 performs four verifier registrations and four commits per process with zero failures; their wall time is charged rather than subtracted. Relative to D0, D2 improves goodput by 21.5× and reduces mean TTFT by 92.90%, closing the earlier layout-only attribution gap.

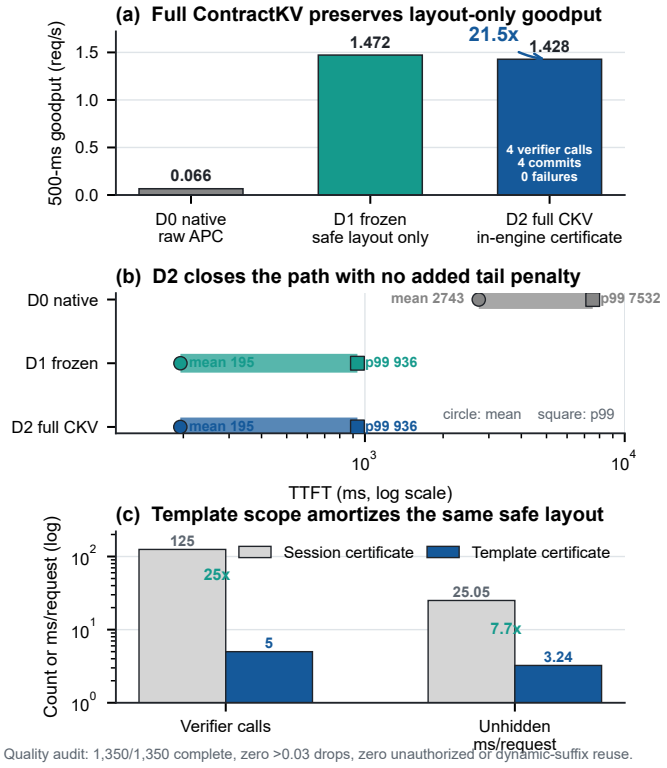


Fig. 4. Verification closure. (a) Absolute 500-ms goodput is 0.066, 1.472, and 1.428 req/s for D0/D1/D2. (b) Same-engine D2 preserves D1's p99. (c) With layout/locality fixed, template scope reduces calls 25× and unhidden exposure 7.7×. All 1,350 quality outputs complete with zero > 0.03 drops or isolation violations.

Template quality audit. Five deployed templates (RULER, HotpotQA, NarrativeQA, Qasper, RepoBench) are each consumed by 10 authorized tenants for 10 turns under three paired fresh processes: 1,350 paired future-turn executions ($5 \times 10 \times 9 \times 3$), clustered by template, tenant, and process. Using frozen task metrics and original budgets, we observe no raw-minus-D2 aggregate score difference, no policy violation above 0.03, no unauthorized reuse, and no scope-invalid admission. Qasper follows fail-closed raw fallback; the remaining families consume shared template certificates with no measured task-quality consequence. This is an empirical unseen-suffix audit of one immutable qualified template across authorized trajectories, not a per-query equivalence proof or merely a marker-bearing load trace.

Scope-granularity ablation. Fig. 4(c) isolates certificate-object granularity. At identical 25-tenant locality, safe layout, and 80% load, template scope reduces verifier calls from 125 to 5 (25×), raises fan-out from 9 to 225 requests per certificate, and reduces unhidden verification cost from 25.05 to 3.24 ms/request (7.7×). Serving work is unchanged: prefix-hit ratios are 97.82% and 98.19%, and mean TTFT, p99, and SLO goodput are statistically indistinguishable. Template scope therefore changes the economics of the safety decision, not the cache opportunity used to obtain performance.

VII. Service-Level Evaluation

We now evaluate ContractKV under multi-tenant service workloads that stress SLO goodput, consolidation density, working-set pressure, and unauthorized-reuse isolation. D0 is native raw-order vLLM APC; D1 is the frozen safe layout without online verification or certificate publication; D2 is full template-scoped ContractKV with the same-engine verifier, certificate publication, and per-request scope check. Arrival timestamps are generated once and replayed identically across D0/D1/D2. The unit-rate trace is exponentially distributed; 100% load is the frozen raw-APC calibration rate, and each cell contains 1,125 future requests over three fresh processes. The load sweep uses Poisson open-loop arrival; the 80% consolidation point adds the frozen ON/OFF burst overlay. Across the service study we execute 288,495 marker-bearing future requests, while task behavior is assessed separately on 2,700 task-bearing raw/ContractKV requests.

A. Service-Level Performance

Fig. 5 reports three complementary views of service-level gain.

Load scaling (Fig. 5a). Under 25-tenant service-template sharing at 80% load and a 500-ms deadline, Qwen3-8B mean TTFT falls by 92.62% (session-cluster 95% CI: 91.68–93.31%). Absolute SLO goodput rises from 0.173 to 0.831 requests/s, a gain of 0.658 requests/s (95% CI: 0.539–0.765) and a 4.81× point multiplier. At 120% load, raw and ContractKV goodput are 0.153 and 1.246 requests/s. A matched Mistral 25-tenant cell reduces mean TTFT by 54.35%, p99 by 29.65%, and new-prefill by 50.75%, with a 0.340 requests/s absolute 500-ms goodput gain (clustered CI: 0.129–0.538). We report absolute curves rather than ratios from near-zero denominators.

Consolidation density (Fig. 5b). Across density-only rows from 10 to 80 tenants under 100% authorized sharing, ContractKV sustains 9.41–11.07 SLO-compliant req/s while raw order remains at 0.90–1.15 requests/s. At 40 tenants, the matched density row is 1.073 versus 11.057 requests/s. A separate 40-tenant ON/OFF replay yields 1.064 versus 10.984 requests/s and raises measured Jain fairness [17] from 0.666 to 1.000; we report that association without claiming a separate fairness mechanism.

Physical deduplication (Fig. 5c). At 80 tenants, peak used KV blocks fall from 530 to 149 (72%) and free blocks rise from 1,856 to 2,237; SLO-compliant throughput is 1.152 requests/s for raw order and 11.033 requests/s for ContractKV. These are allocator-observed physical-block counts rather than a conversion inferred from new-prefill tokens. The released blocks are immediately measurable allocator headroom: a provider-specific pricing model may translate that headroom into cost or margin, but no such economic conversion is assumed here.

B. Operating Region and Boundaries

Fig. 6 characterizes the *operating region*—the conditions under which ContractKV's gains materialize.

Tool-slack sensitivity. Using measured canary latency and live TTFT, we recompute net latency at 0, 500, 1,000, 1,500, 2,000, and 3,000 ms available slack. At zero slack, session-scoped ContractKV remains net-positive by 13.18%, 27.26%,

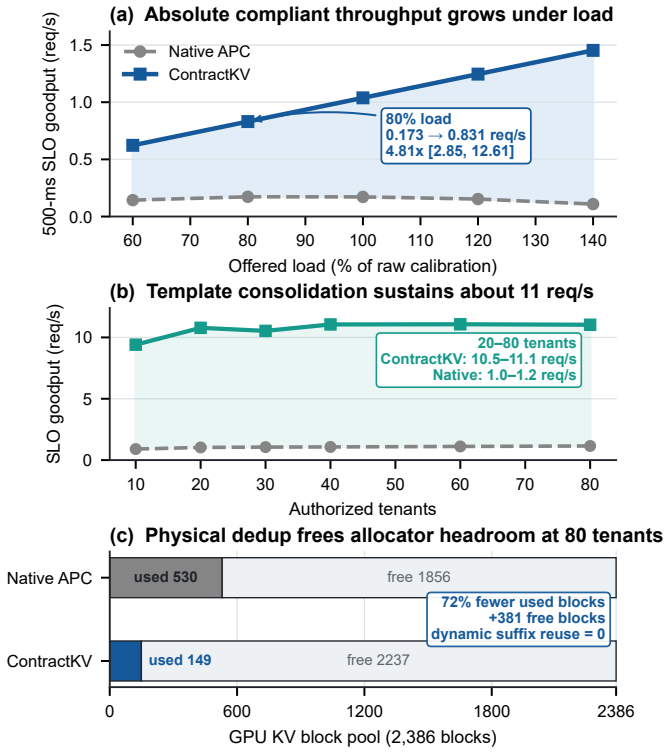


Fig. 5. Service-level benefit. (a) Absolute 500-ms SLO goodput across offered load; the shaded gap is usable compliant capacity, not a ratio with a near-zero denominator. (b) ContractKV sustains roughly 10–11 compliant requests/s across 10–80 tenants. (c) Allocator-observed peak used KV blocks fall from 530 to 149 (72%), creating 381 additional free blocks with zero dynamic-suffix reuse.

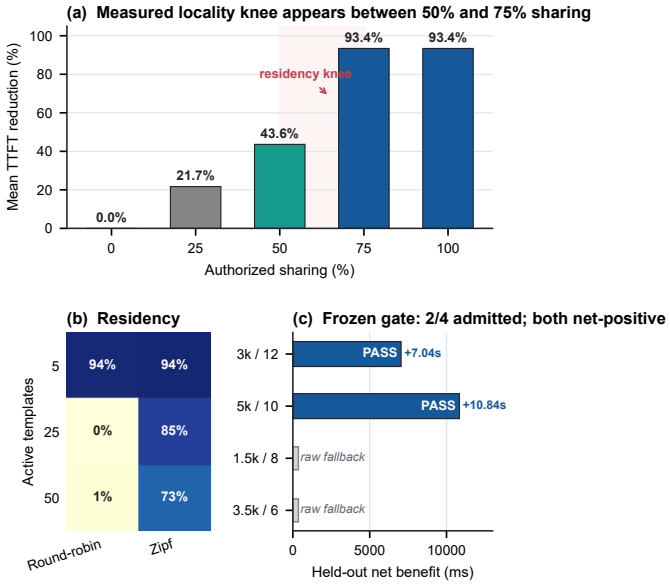


Fig. 6. Operating region. (a) The measured locality knee lies between 50% and 75% sharing. (b) Skew preserves hot templates as the working set grows, whereas round-robin reuse collapses beyond cache capacity. (c) The frozen gate admits two sealed lanes, both net-positive, and routes the remaining two lanes to raw order. Rejected diagnostic outcomes are not interpreted as classification labels.

and 31.20% at concurrency 1, 2, and 4; at 2,000 ms the reductions are 35.99%, 41.66%, and 39.56%. Thus slack expands the benefit but is not assumed free. Low-locality or short-prefix cells can still be negative, while service-template scope reduces unhidden verification to 3.24 ms/request through deployment-object fan-out.

Observed sharing-ratio knee (Fig. 6a). We sweep the fraction of requests authorized under a shared template over 0, 25, 50, 75, and 100%. The measured knee lies between 50% and 75% sharing: once enough requests consume the certified deployment object, its prefix remains resident and TTFT reduction exceeds 85%. This is an operating-region observation for the evaluated trace and cache size, not a universal 75% threshold.

Working-set residency (Fig. 6b). Under Zipf arrival (modeling realistic skewed access), ContractKV retains 73–94% TTFT reduction even at 50 active templates because popular templates remain cache-resident. Under round-robin arrival, gains collapse to zero when reuse distance exceeds cache capacity (25+ templates). The asymmetry confirms that ContractKV's benefit is proportional to the ratio of cache capacity to effective working-set size under the actual access distribution.

Frozen net-benefit admission (Fig. 6c). We freeze the admission rule on development measurements and then evaluate four held-out lanes without changing thresholds. The gate admits the 3,000-token/12-reuse and 5,000-token/10-reuse lanes; both realize positive net savings (+7.04 and +10.84 s per certificate). It routes the shorter 1,500-token/8-reuse and low-fan-out 3,500-token/6-reuse lanes directly to raw order. This is a conservative operating-point controller, not a claim of perfect benefit prediction. A separate zero-locality negative control measures 1.6–8.7% TTFT overhead if compilation is forced, motivating the raw fallback.

TABLE V
SAFETY AND TRANSACTIONAL-CLOSURE EVIDENCE. LIVE SOURCE-HOOK TESTS AND DETERMINISTIC STATE-MACHINE STRESS ARE REPORTED SEPARATELY.

Dimension	Result	Evidence
Live source-hook gate	16/16 pass	12 commit; 4 abort; stable reuse 100%; suffix reuse 0
Concurrent fault probe	10/10 pass	Cancel, incomplete, disconnect, duplicate, isolation
Signed binding stress	15/15 pass	0 false admit/reject; malicious metadata fails closed
Deterministic PSTP stress	325,000 txns	0 invariant failures; 0 aborted states with published blocks
Template quality audit	1,350/1,350	Equal aggregate score; 0 policy/isolation violations
Frozen benefit gate	2/4 lanes admitted	Both admitted lanes are net-positive; non-admitted lanes route raw

C. Safety, Lifecycle, and Economics

Table V consolidates the safety, lifecycle, and verification-economics evidence across all service-level experiments.

The live hook establishes that PASS reaches visible shared blocks and FAIL reaches zero-pollution abort in the real scheduler path. The 325,000-transaction result is a deterministic in-memory concurrency closure for the formal state machine; it is not presented as 325,000 live GPU requests. Across lifecycle changes (model, tokenizer, policy, content, authorization, and combinations), dependent certificates invalidate before reuse and re-register under the new generation. The task-score

aggregate is 0.356 for both raw and D2-template service, and Qasper remains on prospective raw fallback.

VIII. Related Work

Prefix, cross-position, and application-aware reuse. PagedAttention [1] allocates KV blocks, RadixAttention [2] indexes prefixes, and Prompt Cache [4] exposes reusable modules. CacheBlend [5], EPIC [6], CacheSlide [7], and MiniPIC [8] address non-prefix or cross-position reuse through recomputation, positional correction, or position-free cache primitives; Parrot [9] exposes application dependencies. These systems determine how structured state can be reused. ContractKV governs when compiled state may become shared service state, requiring behavioral evidence, composite identity, authorization, lifecycle validity, and atomic publication. DistServe [3] and Mooncake [13] instead optimize serving placement and disaggregation.

KV reduction and prompt construction. H2O [11] and SnapKV [12] retain selected tokens, while CacheGen [10] compresses transferred KV. They optimize within a fixed prompt order. Stable-first prompt construction can expose prefixes but supplies neither behavioral evidence nor atomic publication. ContractKV increases reusable continuity only when a scoped certificate authorizes it, and otherwise preserves raw order.

Service contracts. Classical contracts encode static QoS obligations [18]. ContractKV's runtime certificate authorizes sharing only for a bound model, policy, content, tenant scope, and lifecycle; mismatch fails closed. This evidence-bound service object and its transactional visibility distinguish ContractKV from application prompt engineering.

IX. Discussion and Conclusion

Why and when ContractKV helps. Benefit requires a large stable segment set, repeated authorized consumption, and cache residency. Fig. 2(d) shows that admissible reuse is a model–workflow property: RULER compiles across models, retrieval admits model-dependent levels, and order-sensitive families may freeze at L0. Always-L3 causes 10/225 development violations; prospective qualification and descent remove them. Coverage variation is thus evidence for certification, which extracts admissible reuse and preserves raw behavior elsewhere.

Limitations and artifact scope. ContractKV targets repeated authorized templates; the frozen gate leaves zero-locality or non-admitted lanes raw. Certificates are scoped empirical attestations, not proofs for arbitrary inputs. Cross-model rows charge residual verifier exposure, and the same-engine Qwen closure measures live registration/publication. The evaluation establishes single-cache-domain service behavior rather than multi-node deployment scalability. PSTP is atomic within one vLLM cache domain; a disaggregated pool additionally requires distributed commit, invalidation, and recovery. The public artifact at https://github.com/zhouz4884-coder/contractkv records frozen manifests, trace commitments, runner scripts, baseline patches, machine-readable results, and the CacheSlide common-protocol blocker rather than substituting a proxy number.

Conclusion. ContractKV combines dependency-valid compilation, scoped certification, and transactional publication. Its strongest Qwen3-8B point reduces mean TTFT by 92.62%, improves 500-ms SLO goodput by 4.81×, and uses 72% fewer peak KV blocks at 80 tenants. On Llama, its matched-runtime reduction is 29.16% versus CacheBlend's 26.96%, with 112× fewer observed paired regressions and none among admitted compiled requests. Four configurations remain net-positive with fallback charged; template certificates lower residual exposure 7.7×. Failed or out-of-scope candidates publish no compiled state.

References

[1] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Oper. Syst. Principles (SOSP)*, 2023, pp. 611–626.

[2] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2024, doi: 10.52202/079017-2000.

[3] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 193–210.

[4] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt Cache: Modular attention reuse for low-latency inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[5] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proc. 20th Eur. Conf. Comput. Syst. (EuroSys)*, 2025, doi: 10.1145/3689031.3696098.

[6] J. Hu *et al.*, "EPIC: Efficient position-independent caching for serving large language models," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025, pp. 24391–24402.

[7] Y. Liu *et al.*, "CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving," in *Proc. 24th USENIX Conf. File Storage Technol. (FAST)*, 2026, pp. 83–99.

[8] N. Ordonez and T. Parnell, "MiniPIC: Flexible position-independent caching in $<$ 100 LOC," *arXiv preprint arXiv:2606.13126*, 2026.

[9] C. Lin *et al.*, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 929–945.

[10] Y. Liu *et al.*, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proc. ACM SIGCOMM*, 2024, pp. 138–156.

[11] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[12] Y. Li *et al.*, "SnapKV: LLM knows what you are looking for before generation," in *Proc. NeurIPS*, 2024.

[13] R. Qin *et al.*, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *arXiv preprint arXiv:2407.00079*, 2024.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2023.

[15] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[16] C. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" *arXiv preprint arXiv:2404.06654*, 2024.

[17] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 26, no. 2, pp. 34–35, 1998.

[18] Y. Wang, Q. He, X. Zhang, D. Ye, and Y. Yang, "Efficient QoS-aware service recommendation for multi-tenant service-based systems in cloud," *IEEE Trans. Services Comput.*, vol. 13, no. 6, pp. 1045–1058, 2020, doi: 10.1109/TSC.2017.2761346.

[19] Y. Bai *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2024, pp. 3119–3137.